

A Practical Guide to Quantum Machine Learning

Slide 2: Recap: Key Pillars from Session 1

Pillar 1: Quantum Fundamentals

Qubits: $\alpha|0\rangle + \beta|1\rangle$ (Superposition)

Entanglement: Non-classical correlations (e.g., Bell states)

Gates (X, H, CNOT) & Circuits: Tools for quantum state manipulation

Measurement: Probabilistic collapse to classical outcomes

Pillar 2: The NISQ Context & VQAs

NISQ: Noisy, Intermediate-Scale Quantum devices (our current reality)

VQAs (Variational Quantum Algorithms): Hybrid quantum-classical loops (PQC + Optimizer) like

VQE & QAOA, designed for NISQ.

Pillar 3: QML Introduction & AI-QC Synergy

QML: Using QC to potentially enhance ML; or ML to advance QC.

Focus: QC for ML tasks.

AI is already helping improve quantum hardware and algorithm design.

Core Challenge: Demonstrating practical, real-world quantum advantage for ML.

Today: We build *practical QML strategies* on these pillars.

Slide 3: Setting the Stage for Practical QML

Session 1: The "Why" and "What"

Why quantum computing for ML? (Potential advantages)

What are the basic quantum tools? (Qubits, gates, circuits)

Session 2: The "How" – Getting Practical

How do we encode classical data into quantum states for ML processing?

How are specific QML algorithms (like QSVMs, VQCs) structured?

How are these quantum models actually trained using hybrid approaches?

How can we start implementing and experimenting with simple QML ideas?

How do we evaluate their performance and what are the limitations?

Our Guiding Principles for Today:

Focus on **operational understanding** of QML algorithms.

Emphasize **data representation** (feature maps).

Illustrate with **conceptual workflows** and connections to software tools.

Maintain awareness of **NISQ-era constraints** and possibilities.

Objective: Move from abstract concepts to a more tangible grasp of QML in action.

Slide 4: The Critical First Step: Data Encoding in QML

The Quantum-Classical Interface: QML algorithms operating on classical data **must** first transform that data into a quantum representation.

Why is Encoding So Important?

Information Bridge: It's how classical information enters the quantum processing pipeline.

Feature Space Definition: The encoding method implicitly defines the (often high-dimensional) quantum feature space where the learning algorithm will operate.

Resource Impact: The complexity of encoding (circuit depth, qubit count) directly affects the overall feasibility and resource requirements of the QML algorithm.

Potential for Advantage: A "powerful" encoding (one that creates complex quantum states hard to simulate classically) might be a source of quantum advantage.

Potential Bottleneck: If encoding is inefficient, it can nullify any speedup from subsequent quantum processing.

The "Garbage In, Garbage Out" Principle Applies:

A poorly chosen or implemented encoding can lead to poor QML model performance, regardless of how sophisticated the quantum algorithm itself is.

This is Not an Afterthought: Data encoding (via Quantum Feature Maps) is a central design choice in any QML workflow.

Slide 5: Quantum Feature Maps: $\Phi(x) \rightarrow |\Phi(x)\rangle$ Revisited

The Mechanism of Encoding: A Quantum Feature Map (Φ) transforms classical data x into a quantum state $|\Phi(x)\rangle$.

Input: Classical feature vector $x = [x_1, x_2, \dots, x_d]$

Process: Apply a quantum circuit $U_{\Phi}(x)$ (whose gates may depend on x) to an initial state (e.g., $|0\dots 0\rangle$).

Output: Quantum state $|\Phi(x)\rangle = U_{\Phi}(x)|0\dots 0\rangle$, living in a Hilbert space.

Purpose in QML:

To embed classical data into a quantum system.

To prepare the input for subsequent quantum processing layers (e.g., a variational ansatz in a QNN).

To define a "quantum feature space" whose geometry (e.g., distances, angles between states $|\Phi(x)\rangle$) might be advantageous for learning.

Key Design Considerations for $U_{\Phi}(x)$:

Data Dependence: How are the features x_i incorporated into the circuit (e.g., as rotation angles, phases)?

Entanglement: Does the circuit use entangling gates (like CNOTs) to create correlations between qubits based on input features?

Non-linearity: Can the map create non-linear relationships between the input x and the output state $|\Phi(x)\rangle$?

Depth & Width: How many qubits (width) and gate layers (depth) does $U_{\Phi}(x)$ require?

The "Art" of Feature Map Design: Finding a map that is efficient, expressive, and beneficial for the ML task is a central challenge and research area.

Slide 6: Angle Encoding (Qubit Encoding) - In Detail & Circuit

Strategy: Map classical feature values directly to rotation angles of single-qubit gates.

Input: Classical feature vector $x = [x_{1}, x_{2}, \dots, x_{d}]$.

Process (per feature x_j , on qubit q_j):

1. Optionally, apply an initial gate (e.g., Hadamard H) to q_j to create superposition.
2. Apply a rotation gate $R_{P}(f(x_j))$ to q_j , where:

$P \in \{X, Y, Z\}$ is the axis of rotation.

$f(x_j)$ is a (possibly non-linear) scaling function mapping x_j to an angle (e.g., x_j , πx_j , $\arctan(x_j)$).

Example Circuit (d features, d qubits):

Qubits $q_0, \dots, q_{d-1}$ initialized to $|0\rangle$.

Layer 1 (Optional): H on $q_0, \dots, q_{d-1}$.

Layer 2 (Encoding): $R_{P_0}(f_0(x_0))$ on q_0 , $R_{P_1}(f_1(x_1))$ on q_1 , ..., $R_{P_{d-1}}(f_{d-1}(x_{d-1}))$ on q_{d-1} .

(Further layers with entanglers can be added for more complex feature maps).

Qiskit Example (`RawFeatureVector` circuit element in older Qiskit, or manual construction):

```
`qc.ry(data_point, 0)`
```

```
`qc.ry(data_point, 1)`
```

Pros: Simple, intuitive, uses high-fidelity single-qubit gates.

Cons: Needs d qubits for d features (if 1-to-1 mapping), choice of P and f(x) matters, basic version creates separable (non-entangled) states.

Slide 7: Amplitude Encoding - In Detail & Circuit Concept

Strategy: Encode N classical data values directly into the 2^n amplitudes of an n -qubit state (where $N=2^n$).

Input: Classical N -dimensional vector $x = [x_0, x_1, \dots, x_{N-1}]$,
normalized so $\sum |x_i|^2 = 1$.

Process: Prepare the quantum state:

$$|\psi_x\rangle = x_0|0\dots 00\rangle + x_1|0\dots 01\rangle + \dots + x_{N-1}|1\dots 11\rangle$$

$$|\psi_x\rangle = \sum_{i=0}^{N-1} x_i |i\rangle \text{ (where } |i\rangle \text{ is the } i\text{-th computational basis state).}$$

Qubit Efficiency: Extremely high – only $n = \log_2 N$ qubits needed.

Example: 1024 features ($N=2^{10}$) $\rightarrow$ 10 qubits.

State Preparation Circuit (The Challenge):

While qubit-efficient, creating an *arbitrary* state $|\psi_x\rangle$ with precisely these N amplitudes is generally hard.

General algorithms for arbitrary state preparation can require $O(N \log N)$ or $O(N)$ gates, which is $O(2^n)$ – often negating the qubit efficiency benefit.

Often visualized as a complex network of controlled rotations.

Efficient preparation is only known for specific structured data or if QRAM (Quantum Random Access Memory) is assumed.

Qiskit Example (`initialize` function or `StatePreparation` class):

```
`qc.initialize(normalized_vector, list_of_qubits)`
```

(Qiskit uses decomposition methods to try and build this circuit).

Pros: Most qubit-frugal encoding. Directly utilizes the vastness of Hilbert space.

Cons: State preparation complexity is the major bottleneck for general data. Data normalization required.
Individual amplitude readout is hard.

Slide 8: Basis Encoding - Pros, Cons & Circuit Concept

Strategy: Map discrete classical inputs to unique computational basis states.

Input: Classical data x from a discrete set of M values (e.g., integers 0 to $M-1$, or categories).

Process:

1. Determine n qubits such that $2^n \geq M$.
2. Represent x as an n -bit binary string (x_{binary}).
3. Prepare the quantum state $|x_{\text{binary}}\rangle$.

Example: $M=4$, inputs $\{A,B,C,D\}$. Map $A \rightarrow 00$, $B \rightarrow 01$, $C \rightarrow 10$, $D \rightarrow 11$.

To encode 'C' (10): Prepare $|10\rangle$.

Circuit for State Preparation (e.g., for $|10\rangle$ on 2 qubits):

q_0 : $|0\rangle \rightarrow X\text{-gate} \rightarrow |1\rangle$

q_1 : $|0\rangle \rightarrow I\text{-gate (no change)} \rightarrow |0\rangle$

Result: $|10\rangle$

Pros:

Very simple to understand and implement.

State preparation is efficient and exact for a given target basis state (just X-gates on $|0\dots 0\rangle$).

No information loss for the discrete values being encoded.

Cons:

Inefficient in qubit use if M is large ($n = \lceil \log_2 M \rceil$ qubits).

Does not use superposition to encode features of a *single* data point.

Distances between classical values may not be reflected in Hamming distances of basis states (e.g., 3 vs 4).

Primary Use: Oracles in algorithms like Grover's, Deutsch-Jozsa; encoding discrete/categorical features in QML if the number of categories is small.

Slide 9: Parameterized Quantum Circuits (PQCs) as Feature Maps

Advancing Beyond Basic Encodings: We can construct more expressive and potentially powerful feature maps using Parameterized Quantum Circuits (PQCs).

The Concept:

The classical data vector $x = [x_1, \dots, x_d]$ provides the **parameters** for a pre-defined PQC structure (the "feature map ansatz").

This data-driven PQC, $U_{\Phi}(x)$, acts on an initial state (e.g., $|0\dots 0\rangle$).

The output state, $|\Phi(x)\rangle = U_{\Phi}(x)|0\dots 0\rangle$, *is* the quantum feature representation.

Typical Structure of PQC Feature Maps:

Data Encoding Layers: Often use single-qubit rotation gates ($R_P(f(x_i))$) to embed features, similar to angle encoding.

Entangling Layers: Crucially, these are interleaved with layers of entangling gates (e.g., CNOTs, CZs, controlled rotations) that create correlations between qubits based on the input data.

Example: Qiskit's `ZZFeatureMap` (Conceptual Structure)

1. Initial Layer: Hadamard gates on all qubits ($H^{\otimes n}$).

2. Data Mapping Block (repeated `reps` times):

Single-qubit phase rotations encoding data: e.g., $R_z(\phi(x_i))$ on qubit i .

Entangling block: e.g., applying $e^{-ig(x_i, x_j)}$

$Z_i Z_j$ terms between pairs of qubits (decomposable into CNOTs and single-qubit gates). The function `g` can introduce non-linear feature interactions.

Benefits:

Controlled Entanglement: Can explicitly design entanglement based on data features.

Non-linearity: The interplay of data-dependent rotations and entanglers can create highly non-linear mappings to Hilbert space.

Tunable Complexity: The depth (`reps`) and structure can be adjusted.

Potential for Richer Kernels: These maps are often designed with quantum kernel estimation in

Slide 10: Choosing a Feature Map: Considerations & Trade-offs

Recap: No "Silver Bullet" Encoding. The optimal feature map is problem, data, and hardware dependent.

Key Decision Factors & Trade-offs:

1. Expressivity:

Pro: Can capture complex patterns, potentially leading to better model performance if the Hilbert space geometry is advantageous.

Con: Can lead to barren plateaus if too unstructured/deep, making training hard. May require more resources.

2. Resource Cost (Qubits & Circuit Depth):

Pro (Low Cost): More feasible on NISQ devices, less error accumulation.

Con (High Cost): May limit problem size or lead to noisy results. Amplitude encoding is qubit-frugal but depth-costly for preparation.

3. "Quantumness" / Classical Simulability:

Pro (High Quantumness): May offer genuine quantum advantage if the feature space is hard to simulate classically (relevant for kernels).

Con (Low Quantumness/Easily Simulable): Less likely to provide advantage over classical methods.

4. Problem-Data Alignment:

Pro: Feature map exploits known symmetries or structure in the data, leading to better learning.

Con: Mismatched feature map may obscure important patterns.

5. Computational Cost of Encoding:

Pro (Fast Encoding): State preparation doesn't become the bottleneck.

Con (Slow Encoding): Negates any downstream processing speedups.

6. Interpretability (Often Low for Complex Maps):

While not always a primary goal, understanding **what** features the map extracts can be difficult.

Current Approach: Often empirical, guided by heuristics, and an active research area. Start simple, add

Slide 11: The Challenge of "Loading" Data Efficiently (Reiteration & QRAM)

Recap: The Data Input Bottleneck: A major hurdle for many QML algorithms, especially those promising speedups based on processing large N-dimensional vectors (e.g., HHL, some forms of amplitude encoding).

The "QRAM Problem":

Quantum Random Access Memory (QRAM) is a hypothetical device that could efficiently prepare a superposition of N data entries, queried by a quantum address register.

$| \text{address} \rangle | 0 \rangle \rightarrow | \text{address} \rangle | \text{data}_{\text{address}} \rangle$ OR $\sum | \text{address} \rangle | 0 \rangle \rightarrow \sum | \text{address} \rangle | \text{data}_{\text{address}} \rangle$

If QRAM existed and was efficient (e.g., $O(\log N)$ access time), it would unlock many QML algorithms.

Current Status: Building scalable, efficient QRAM is a formidable research challenge in itself, with no clear practical realization yet. Most QRAM proposals have significant resource overheads.

Impact on Amplitude Encoding: Amplitude encoding's promise of $\log^2 N$ qubit efficiency for N features often implicitly relies on efficient state preparation, which QRAM would provide. Without it, preparing arbitrary amplitude-encoded states is generally slow.

Impact on Other Algorithms: Any QML algorithm that needs to "look up" or load many classical data points into a quantum state quickly faces this issue.

NISQ Era Focus: Given QRAM unavailability, current practical QML often focuses on:

Encoding smaller numbers of features where state preparation is manageable (e.g., angle encoding for a few features per data point).

Feature maps that are circuits applied to a simple initial state, where data influences gate parameters.

Problems where the "quantum part" processes features in a way that doesn't require loading the entire dataset into one massive quantum state.

This is a Key Differentiator between Theoretical QML Power and Practical NISQ Application.

Slide 12: Classical Support Vector Machines (SVMs) - A Quick Refresher

A Powerful Classical Supervised Learning Algorithm: SVMs are widely used for classification (and regression) tasks.

Goal (for Classification): To find an optimal hyperplane (a line in 2D, a plane in 3D, a hyperplane in higher dimensions) that best separates data points belonging to different classes in the feature space.

Maximizing the Margin:

The "optimal" hyperplane is the one that has the largest margin (distance) to the nearest data points of any class. These nearest points are called "support vectors."

A larger margin generally leads to better generalization performance (better classification on unseen data).

Linear SVMs: Find a linear separator.

Equation of hyperplane: $w \cdot x - b = 0$ (w is weight vector, b is bias).

Non-Linear SVMs & The Kernel Trick:

What if data is not linearly separable in the original feature space?

SVMs can use the "kernel trick" to handle this.

Support Vectors: The data points that lie closest to the hyperplane and influence its position and orientation. Only these points are critical for defining the separator.

Strengths: Effective in high-dimensional spaces, memory efficient (uses subset of training points – support vectors), versatile with different kernels.

Slide 13: The Kernel Trick in Classical SVMs

Challenge: Non-Linearly Separable Data: Many real-world datasets cannot be separated by a simple linear hyperplane in their original feature space.

The Idea: Map to a Higher-Dimensional Space:

Project the data into a higher-dimensional feature space where it *might* become linearly separable.

Example: Data points on a 1D line $[x]$ might become separable in 2D $[x, x^2]$.

The Kernel Trick - "Magic":

SVMs only need to calculate dot products (inner products) between data points in this higher-dimensional space, not the coordinates of the points themselves.

A **kernel function** $K(x_i, x_j)$ directly computes this inner product in the high-dimensional feature space, $K(x_i, x_j) = \Phi(x_i) \cdot \Phi(x_j)$, where Φ is the mapping to the high-dimensional space.

This avoids the computationally expensive step of explicitly transforming every data point into the high-dimensional space.

Common Classical Kernels:

Polynomial Kernel: $K(x_i, x_j) = (\gamma x_i \cdot x_j + r)^d$

Radial Basis Function (RBF) Kernel: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ (maps to an infinite-dimensional space).

Sigmoid Kernel.

How it Works in SVM Training: The SVM optimization algorithm is formulated using only these kernel evaluations between pairs of training points (the Gram matrix).

Power: Allows linear algorithms (like SVMs) to learn non-linear decision boundaries efficiently.

Slide 14: Introducing Quantum Kernel Methods

The QML Analogy to the Classical Kernel Trick: Can we use quantum computers to define and compute powerful kernel functions?

Core Idea of Quantum Kernel Methods:

1. **Quantum Feature Map (Φ):** Use a quantum circuit $U_{\Phi}(x)$ to map classical data points x into quantum states $|\Phi(x)\rangle$ in a (potentially very large) quantum Hilbert space.

$$|\Phi(x)\rangle = U_{\Phi}(x)|0\dots 0\rangle$$

2. **Quantum Kernel Estimation:** Use a quantum computer to estimate the "similarity" (inner product or fidelity) between these quantum feature states.

$$K_{\text{quantum}}(x_i, x_j) = |\langle \Phi(x_i) | \Phi(x_j) \rangle|^2 \quad (\text{This is a common definition, related to fidelity}).$$

3. **Classical ML Algorithm:** Feed this quantumly computed kernel matrix K_{quantum} into a classical kernel-based machine learning algorithm, like a classical SVM.

Hybrid Quantum-Classical Approach:

Quantum computer: Used for the feature mapping and kernel entry estimation.

Classical computer: Used for the rest of the ML algorithm (e.g., SVM optimization).

Potential for Quantum Advantage:

If the quantum feature map $\Phi(x)$ can efficiently create a feature space that is hard for classical computers to access or simulate...

And if this quantum feature space provides better separability or captures more relevant data structures for the learning task...

Then, a quantum kernel might lead to better ML model performance (e.g., higher accuracy) than any classical kernel.

This is a prominent approach for NISQ-era QML.

Slide 15: Defining a Quantum Kernel: $K(x_{\text{i}}, x_{\text{j}}) = |\langle \Phi(x_{\text{i}}) | \Phi(x_{\text{j}}) \rangle|^2$

The Heart of the Method: The quantum kernel function $K(x_{\text{i}}, x_{\text{j}})$ measures the similarity between two classical data points x_{i} and x_{j} *in the quantum feature space*.

Step 1: Quantum Feature Mapping (Recap)

$$x_{\text{i}} \rightarrow U_{\Phi}(x_{\text{i}}) |0\dots 0\rangle = |\Phi(x_{\text{i}})\rangle$$

$$x_{\text{j}} \rightarrow U_{\Phi}(x_{\text{j}}) |0\dots 0\rangle = |\Phi(x_{\text{j}})\rangle$$

The same feature map circuit U_{Φ} is used, but parameterized by x_{i} and x_{j} respectively.

Step 2: Calculating the Overlap (Inner Product)

The inner product $\langle \Phi(x_{\text{i}}) | \Phi(x_{\text{j}}) \rangle$ is a complex number whose magnitude indicates how much the two quantum states "overlap" or "align."

If $|\Phi(x_{\text{i}})\rangle$ and $|\Phi(x_{\text{j}})\rangle$ are identical (up to a global phase), $|\langle \Phi(x_{\text{i}}) | \Phi(x_{\text{j}}) \rangle| = 1$.

If they are orthogonal, $\langle \Phi(x_{\text{i}}) | \Phi(x_{\text{j}}) \rangle = 0$.

Step 3: The Kernel Value – Squared Overlap (Fidelity-like)

$$K(x_{\text{i}}, x_{\text{j}}) = |\langle \Phi(x_{\text{i}}) | \Phi(x_{\text{j}}) \rangle|^2$$

This value is always real and between 0 and 1.

It's a measure of "closeness" or "similarity." $K(x, x) = 1$.

Why this definition?

It naturally arises from how we can estimate overlaps on a quantum computer.

It ensures the resulting kernel matrix is positive semi-definite (a requirement for SVMs), provided the mapping is to a Hilbert space.

The Power Lies in $\Phi(x)$: The "quantumness" or "richness" of the kernel comes entirely from the choice of the quantum feature map $U_{\Phi}(x)$. If $U_{\Phi}(x)$ creates complex, entangled states, the resulting kernel might capture similarities that classical kernels miss.

Slide 16: Estimating Quantum Kernels on a QPU (e.g., SWAP Test, other methods)

Challenge: We need a quantum circuit to estimate $K_{ij} = |\langle \Phi(x_i) | \Phi(x_j) \rangle|^2$.

Method 1: The SWAP Test Circuit (Historical/Conceptual)

Uses an ancilla qubit, initialized to $|0\rangle$ and H-gated.

Prepare $|\Phi(x_i)\rangle$ on one register of qubits.

Prepare $|\Phi(x_j)\rangle$ on a second register of qubits.

Apply a controlled-SWAP operation: the ancilla qubit controls the SWAP of the two registers.

Apply another H-gate to the ancilla qubit.

Measure the ancilla qubit in the Z-basis.

The probability of measuring the ancilla as $|0\rangle$ is $P(0) = \frac{1}{2} + \frac{1}{2} |\langle \Phi(x_i) | \Phi(x_j) \rangle|^2$.

So, $K_{ij} = 2 * P(0) - 1$.

Requires two separate state preparations and a C-SWAP gate (which can be costly).

Method 2: "Compute-Uncompute" / Inversion Test (More NISQ-Friendly for some cases)

Circuit: $U_{\Phi}^\dagger(x_j) U_{\Phi}(x_i) |0\dots 0\rangle$

This prepares $|\Phi(x_i)\rangle$ and then tries to "unprepare" it using the inverse of the feature map for x_j .

If $x_i = x_j$, the state returns to $|0\dots 0\rangle$.

The probability of measuring the all-zeros state $|0\dots 0\rangle$ at the end is $P(|0\dots 0\rangle) = |\langle 0\dots 0 |$

$U_{\Phi}^\dagger(x_j) U_{\Phi}(x_i) |0\dots 0\rangle|^2$

$= |\langle \Phi(x_j) | \Phi(x_i) \rangle|^2 = K_{ij}$.

Requires only one register of qubits for state prep, and the ability to run

$U_{\Phi}^\dagger(x_j)$.

Other Approaches: Various interferometric circuits exist. Some SDKs (like Qiskit) have built-in kernel estimation methods that optimize this.

Measurement Statistics: All methods require running the estimation circuit many times (shots) to get good statistical estimates of the probabilities, and thus K_{ij} .

Slide 17: The Quantum Support Vector Machine (QSVM) Algorithm Workflow

A Hybrid Quantum-Classical Algorithm:

Goal: To perform classification using a classical SVM but with a kernel computed by a quantum computer.

Workflow Steps:

1. Choose a Quantum Feature Map $\Phi(x)$:

This is a critical design choice (e.g., `ZZFeatureMap`, `PauliFeatureMap` from Qiskit, or a custom PQC).

It defines how classical data x is encoded into quantum states $|\Phi(x)\rangle$.

2. Quantum Kernel Matrix Computation (on QPU + CPU):

For every pair of training data points ($x_{i\langle\text{sub}\rangle}$, $x_{j\langle\text{sub}\rangle}$) in your training set:

QPU: Execute a quantum circuit (e.g., compute-uncompute method) to estimate $K_{ij\langle\text{sub}\rangle} = |\langle\Phi(x_{i\langle\text{sub}\rangle})|\Phi(x_{j\langle\text{sub}\rangle})\rangle|^2$. This requires many shots per pair.

CPU: Collect all $K_{ij\langle\text{sub}\rangle}$ values to form the full $N_{\text{train}\langle\text{sub}\rangle} \times N_{\text{train}\langle\text{sub}\rangle}$ quantum kernel matrix (Gram matrix).

3. Classical SVM Training (on CPU):

Feed the quantum kernel matrix and the training labels $y_{i\langle\text{sub}\rangle}$ into a classical SVM solver.

The classical SVM solver finds the support vectors and decision boundary *in the quantum feature space*, but it does so implicitly using only the kernel matrix.

This step is purely classical.

4. Prediction for New Data (Hybrid):

For a new, unseen data point $x_{\text{new}\langle\text{sub}\rangle}$:

QPU+CPU: Compute the kernel entries $K(x_{\text{new}\langle\text{sub}\rangle}, x_{\text{sv}\langle\text{sub}\rangle})$ between $x_{\text{new}\langle\text{sub}\rangle}$ and each of the support vectors $x_{\text{sv}\langle\text{sub}\rangle}$ found during training.

CPU: Use these kernel values and the SVM model parameters (alphas, bias

Slide 18: Potential Advantages of Quantum Kernels (Richer Feature Spaces?)

The Core Hypothesis: Quantum feature maps $\Phi(x)$ can project classical data into extremely high-dimensional Hilbert spaces that are difficult or impossible for classical computers to access or simulate efficiently.

"Richer" Feature Spaces:

These quantum feature spaces might possess a more complex geometry.

This complexity (e.g., due to entanglement and interference effects in the feature map $U_{\Phi}(x)$) could allow for better separation of data that is highly non-linear in the original space.

Potential for Better Generalization:

If a quantum kernel captures the underlying structure of the data more effectively than classical kernels, the resulting SVM might achieve higher accuracy on unseen data.

When Might Quantum Kernels Outperform Classical Kernels?

If the feature map $U_{\Phi}(x)$ is "hard" to simulate classically: If the quantum states $|\Phi(x)\rangle$ and their inner products could be easily calculated by a classical computer, there would likely be no quantum advantage. The advantage hinges on the feature map doing something uniquely quantum and computationally difficult for classical methods.

If the resulting quantum feature space is genuinely more suitable for the specific dataset and classification task.

No Guarantee of Advantage:

Not all quantum feature maps will lead to better performance. A poorly designed quantum kernel could perform worse than a good classical kernel (e.g., RBF).

The advantage is problem-dependent and feature-map-dependent.

Active Research: Identifying classes of problems and feature maps where quantum kernels offer a demonstrable advantage is a major focus of QML research. This includes understanding how to avoid feature maps that are classically simulable.

Slide 19: Practical Implementation: Qiskit's `QuantumKernel` and `QSVC`

Bringing Quantum Kernels to Life with Software: Libraries like Qiskit provide tools to implement and experiment with quantum kernel methods.

Qiskit Machine Learning Module:

`FeatureMap` classes: Provide pre-built PQC circuits for data encoding.

`ZZFeatureMap`: Encodes data using single-qubit Z-rotations and ZZ-interactions (entanglement). Parameters: `feature\_dimension`, `reps` (depth), `entanglement` (e.g., 'linear', 'full').

`PauliFeatureMap`: More general, uses Pauli gates and specified entanglement.

`NLocal` circuits can also be used for custom feature maps.

`QuantumKernel` class:

Takes a `FeatureMap` object and a quantum backend (simulator or real QPU) as input.

Provides methods to compute the kernel matrix (the Gram matrix) between sets of data points by executing the necessary quantum circuits to estimate $|\langle \Phi(x_{_i}) | \Phi(x_{_j}) \rangle|^2$.

`QSVC` class (Quantum Support Vector Classifier):

A scikit-learn compatible classifier.

Takes a `QuantumKernel` object as input.

Handles the classical SVM training part using the quantumly computed kernel.

Provides familiar `.fit(X\_train, y\_train)` and `.predict(X\_test)` methods.

Conceptual Workflow in Qiskit:

1. `fm = ZZFeatureMap(feature_dimension=num_features, reps=depth)`
2. `kernel = QuantumKernel(feature_map=fm, quantum_instance=backend)`
3. `qsvc = QSVC(quantum_kernel=kernel)`
4. `qsvc.fit(training_data, training_labels)`
5. `predictions = qsvc.predict(test_data)`

Allows for experimentation with different feature maps and backends.

Slide 20: Challenges and Current Research in Quantum Kernels

Promise vs. Practicality: While elegant, quantum kernels face challenges for demonstrating broad quantum advantage.

1. Classical Simulability of Kernels:

A key question: Can the specific quantum kernel being computed be efficiently simulated or approximated by a classical computer?

If yes, then no quantum advantage is expected for that kernel.

Research focuses on identifying feature maps that generate kernels believed to be "hard" to compute classically.

2. Data Dependence of Advantage:

Any advantage is likely highly dependent on the specific dataset. A quantum kernel might outperform classical on one dataset but underperform on another.

3. Computational Cost of Kernel Matrix Estimation:

Estimating each K_{ij} requires many shots on a QPU.

For N_{train} training points, building the full $N_{\text{train}} \times N_{\text{train}}$ kernel matrix requires $\sim N_{\text{train}}^2 / 2$ QPU evaluations, which can be very time-consuming for large datasets.

4. "Curse of Dimensionality" in Feature Space:

While quantum feature maps can access high-dimensional Hilbert spaces, these spaces can also suffer from concentration effects (e.g., all inner products becoming very similar, leading to a trivial kernel) if not designed carefully, especially for certain types of feature maps and data distributions. This can lead to poor trainability or generalization.

5. Noise on NISQ Devices:

Errors in QPU execution will lead to noisy estimates of K_{ij} , which can degrade the performance of the classical SVM.

Active Research Areas:

Designing "provably quantum" feature maps.

Understanding which datasets benefit most.

Slide 21: Example: QSVM for a Simple Classification Task (Conceptual)

Goal: Classify 2D data points into two classes (e.g., circles vs. crosses).

Steps (Conceptual, using Qiskit-like flow):

1. **Generate/Load Data:** Create a simple, non-linearly separable 2D dataset (e.g., scikit-learn's ``make_circles`` or ``make_moons``).

2. **Define Quantum Feature Map:**

``fm = ZZFeatureMap(feature_dimension=2, reps=2, entanglement='linear')``

This maps each 2D point $x = [x_1, x_2]$ into a multi-qubit quantum state $|\Phi(x)\rangle$.

3. **Instantiate Quantum Kernel:**

``kernel = QuantumKernel(feature_map=fm, quantum_instance=Aer.get_backend('qasm_simulator'))``

(Using ``qasm_simulator`` for shot-based estimation, more realistic for NISQ).

4. **Train QSVC Model:**

``qsvc = QSVC(quantum_kernel=kernel)``

``qsvc.fit(X_train, y_train)``

The ``fit`` method will trigger the QPU (simulator here) to compute all pairwise kernel entries K_{ij} for the training data.

5. **Evaluate Model:**

``accuracy = qsvc.score(X_test, y_test)``

Plot decision boundary (can be complex due to the quantum kernel).

What's Happening Quantumly:

For each K_{ij} , circuits

$U_{\Phi}^\dagger(x_j)U_{\Phi}(x_i)|0\rangle$ are run many times.

The probability of measuring $|00\dots 0\rangle$ gives K_{ij} .

Expected Outcome: The QSVM, using the ``ZZFeatureMap`` kernel, might find a non-linear decision boundary that successfully separates the classes in the original 2D space, by implicitly working in a higher-dimensional quantum feature space.

Slide 22: Introduction to VQCs/QNNs for Classification

Beyond Kernels: End-to-End Quantum Classifiers: Variational Quantum Classifiers (VQCs), often called Quantum Neural Networks (QNNs), offer another paradigm for QML.

Core Idea: Train a Parameterized Quantum Circuit (PQC) directly to perform a classification task.

- The PQC acts like a trainable quantum model.

- Classical data is encoded into the PQC.

- Measurements on the output of the PQC are interpreted as class predictions.

- Parameters of the PQC are optimized using a hybrid quantum-classical loop to minimize a classification loss function.

Analogy to Classical Neural Networks:

- Input Layer:** Data encoding part of the PQC (feature map).

- Hidden Layers:** The parameterized/variational part of the PQC (the ansatz).

- Output Layer:** Measurement and classical post-processing to get a class label/probability.

- Weights/Biases:** The tunable parameters (e.g., rotation angles) within the PQC.

Key Difference from Quantum Kernels:

- Quantum Kernels: QPU computes kernel matrix; classical SVM does the learning.

- VQCs/QNNs: QPU *is* the model (in part); its parameters are learned directly for the classification task. The learning loop involves both QPU and CPU.

Well-Suited for NISQ: VQCs are designed with shallow circuits and hybrid optimization, making them a primary focus for near-term QML research.

Slide 23: General Architecture of a VQC/QNN

A Modular Structure: Most VQCs/QNNs can be thought of as having three main components:

1. State Preparation / Data Encoding Circuit (Feature Map):

$$U_{\Phi}(x)$$

Encodes the classical input data vector x into an initial quantum state $|\Phi(x)\rangle$.

Can be fixed or have some learnable parameters itself (though often fixed for simplicity).

Uses techniques like angle encoding, or more complex PQC-based feature maps.

2. Variational Circuit / Ansatz (The "Model"):

$$U_W(\theta)$$

A Parameterized Quantum Circuit with learnable parameters θ (the "weights").

This is where the main "learning" occurs. It transforms the encoded state: $|\psi(x, \theta)\rangle = U_W(\theta) |\Phi(x)\rangle$.

Structure often consists of layers of single-qubit rotations and entangling gates.

3. Measurement & Classical Post-Processing (The "Output"):

Measure one or more qubits of $|\psi(x, \theta)\rangle$.

Convert measurement outcomes (e.g., expectation value of a Pauli operator, probability of measuring $|0\rangle$) into a classical prediction (e.g., class label, score).

Overall Operation: $|\text{Prediction}\rangle = \text{ClassicalPostProcess}(\text{Measure}(U_W(\theta) U_{\Phi}(x) |0\dots 0\rangle))$

Training: The parameters θ in $U_W(\theta)$ (and sometimes in $U_{\Phi}(x)$) are optimized to minimize a task-specific loss function.

Slide 24: Data Encoding Layer within the VQC/QNN

Role: To embed classical input features x into the initial state of the quantum circuit.

Common Approaches (as Feature Maps):

Angle Encoding:

Features x_i directly set rotation angles of R_x , R_y , or R_z gates.

Often applied after an initial layer of Hadamard gates.

Example: `qc.h(qubit); qc.rz(feature_value, qubit);`

Dense Angle Encoding: Multiple features might be combined to set angles, or features might be encoded in multiple rotations on the same qubit.

Amplitude Encoding (Less Common for VQC inputs due to preparation complexity):

Could theoretically be used if efficient state prep for the specific data is possible.

Integration with Variational Layers:

The output of this data encoding layer $|\Phi(x)\rangle$ becomes the input to the subsequent parameterized variational layers $U_W(\theta)$.

Fixed vs. Learnable Encoding:

Fixed: The structure and any non-data parameters of the encoding circuit are pre-defined and do not change during training (most common).

Learnable (More Advanced): The encoding circuit itself might contain some trainable parameters alongside the data-dependent ones, which are optimized during the main training loop. This blurs the line between feature engineering and model training.

Impact: The choice of encoding significantly influences the "landscape" that the variational layers explore and the overall capacity of the QNN.

Slide 25: The Variational Ansatz Layer (Learnable Part)

The Core "Learning Engine" of the VQC/QNN: The variational circuit, or ansatz $U_W(\theta)$, is where the model's learnable parameters θ reside.

Structure: Typically composed of repeating layers. Each layer usually contains:

1. Parameterized Single-Qubit Rotations:

$R_x(\theta_{ij})$, $R_y(\theta_{ij})$,
 $R_z(\theta_{ij})$ applied to each qubit i in layer j .
These θ_{ij} are the primary learnable parameters.

2. Entangling Block:

A fixed set of entangling gates (e.g., CNOTs, CZs) applied between qubits.
Common patterns: linear chain of CNOTs, circular entanglement, all-to-all (if hardware allows), or more complex patterns.
Purpose: To create correlations between qubits and allow information to propagate across the quantum register, enabling the learning of complex functions.

Depth of the Ansatz (`reps` or `L` layers):

More layers generally mean more parameters and greater expressivity (ability to represent more complex functions/states).
However, deeper circuits are more prone to noise on NISQ devices and can lead to trainability issues (e.g., barren plateaus).

Design Principles ("Ansatz Design"):

Hardware-Efficient Ansätze: Use gates native to the target QPU and respect its connectivity to minimize SWAP overhead and depth.

Problem-Inspired Ansätze: Structure reflects symmetries or properties of the problem (less common for generic classifiers, more for VQE/QAOA).

Expressivity vs. Trainability Trade-off: A key consideration.

The parameters θ are iteratively updated by a classical optimizer based on a defined loss function.

Slide 26: Measurement Strategies for Classification Output

Extracting a Classical Prediction: After the data-encoded state $|\Phi(x)\rangle$ passes through the variational ansatz $U_{W(\theta)}$ to become $|\psi(x,\theta)\rangle$, we need to measure it to get a classical output that can be used for classification.

Common Measurement Approaches for VQCs:

1. Expectation Value of a Pauli Operator (Often Z):

Measure the expectation value of a Pauli operator (e.g., Pauli Z) on one (or a few) specific output/ancilla qubit(s).

$\langle Z_k \rangle = P(\text{qubit } k \text{ is } 0) - P(\text{qubit } k \text{ is } 1)$. This value is between -1 and +1.

This real-valued output can then be:

- Directly used as a score (e.g., for binary classification, >0 is class A, <0 is class B).

- Passed through a classical activation function (e.g., sigmoid) to get a probability.

2. Probability of a Specific Basis State:

Measure all (or a subset of) qubits in the computational basis.

Calculate the probability of obtaining a specific bit string (e.g., $P(|0\dots 0\rangle)$ or $P(|1\rangle)$ on a specific qubit)).

This probability (between 0 and 1) can be used as the model's output.

3. Multiple Qubit Measurements & Parity Mapping:

Measure multiple qubits.

Map the resulting multi-bit string to a class label using a pre-defined function (e.g., based on the parity of the bit string).

Number of Shots:

Since measurements are probabilistic, each circuit execution (for a given x and θ) is typically run for many "shots" (e.g., 1000-8000 times).

The classical output (expectation value or probability) is estimated from the statistics of these shots.

Slide 27: Defining the Cost/Loss Function for Training VQCs

Guiding the Learning Process: The cost function (or loss function) L quantifies how "wrong" the VQC's current predictions are compared to the true labels.

Input to Loss Function:

True labels y from the training dataset.

Predictions $\hat{y}(x, \theta)$ from the VQC (derived from measurements of $|\psi(x, \theta)\rangle$).

Goal of Training: To find the parameters θ of the variational ansatz $U_{W}(\theta)$ that **minimize** this loss function.

Common Loss Functions for Classification VQCs:

Mean Squared Error (MSE):

$$L(\theta) = (1/M) \sum_{i=1}^M (y_{i} - \hat{y}(x_{i}, \theta))^2$$

Often used if $\hat{y}$ is a continuous value (e.g., an expectation value mapped to a score).
 y_{i} might be +1/-1 for binary classes.

Cross-Entropy Loss (Binary or Categorical):

More common if $\hat{y}$ is interpreted as a class probability.

Binary Cross-Entropy: $L(\theta) = -(1/M) \sum [y_{i} \log(\hat{y}_{i}) + (1 - y_{i}) \log(1 - \hat{y}_{i})]$

Requires $\hat{y}_{i}$ to be in (0,1) (e.g., output of a sigmoid function applied to an expectation value).

Hinge Loss (Used in SVMs): Can also be adapted if the VQC output is a score.

The Loss Function is Classical: It's calculated on the classical computer using classical predictions derived from quantum measurements.

Differentiability: If using gradient-based optimizers, the loss function (and thus the way predictions $\hat{y}$ depend on θ) must be differentiable with respect to θ . This is where techniques like the parameter-shift rule for quantum gradients become essential.

Slide 28: Training VQCs: The Hybrid Quantum-Classical Loop

Iterative Optimization: Training a VQC is an iterative process that closely mirrors the general VQA workflow.

The Loop:

1. **Initialize Parameters θ :** Start with an initial guess for the PQC parameters (CPU).
2. **For each batch of training data $\{x_{_i}, y_{_i}\}$:**
 - a. **Quantum Forward Pass (QPU):**

For each $x_{_i}$: Encode $x_{_i} \rightarrow |\Phi(x_{_i})\rangle$.
Apply variational circuit: $U_{_W}(\theta) |\Phi(x_{_i})\rangle = |\psi(x_{_i}, \theta)\rangle$.
Perform measurements.
 - b. **Classical Post-Processing & Prediction (CPU):**

Convert measurement outcomes to prediction $\hat{y}(x_{_i}, \theta)$.
 - c. **Loss Calculation (CPU):**

Compute loss $L(\{\hat{y}(x_{_i}, \theta)\}, \{y_{_i}\})$ for the batch.
3. **Gradient Estimation (QPU + CPU, if gradient-based optimizer):**

Estimate gradients $\nabla_{_{\theta}} L$ (e.g., using parameter-shift rule, which involves more QPU calls).
4. **Parameter Update (CPU):**

Classical optimizer uses the loss (and gradients, if available) to compute updated parameters $\theta_{_{new}} = \theta - \eta \nabla_{_{\theta}} L$ (for gradient descent).
5. **Repeat:** Go back to Step 2 with $\theta_{_{new}}$ until convergence (loss plateaus, accuracy goals met) or max iterations reached.

Key: Tight integration between QPU (evaluating parameterized circuits) and CPU (calculating loss, optimizing parameters).

Slide 29: Gradient Estimation for VQCs (Parameter-Shift Rule)

Need for Gradients: Many powerful classical optimizers (SGD, Adam, etc.) are gradient-based. To use them for VQCs, we need $\partial L / \partial \theta_k$ for each parameter θ_k .

Challenge: The PQC $U_W(\theta)$ is on a quantum computer. How do we get gradients of a cost $C(\theta)$ (derived from expectation values) that involves QPU execution?

Parameter-Shift Rule: An Analytic Quantum Gradient

A remarkable result for a common class of parameterized quantum gates (e.g., single-qubit rotations $R_P(\theta) = e^{-i\theta P/2}$ where $P^2 = I$).

The exact gradient of an expectation value $\langle M \rangle_{\theta} = \langle 0 | U^\dagger(\theta) M U(\theta) | 0 \rangle$ with respect to a parameter θ_k (e.g., an angle in a specific rotation gate) can be found by:

$$\frac{\partial \langle M \rangle_{\theta}}{\partial \theta_k} = \frac{r}{2} * [\langle M \rangle_{\theta_k+s} - \langle M \rangle_{\theta_k-s}]$$

where:

• s is a specific shift (often $\pi/2$ or $\pi/4r$ depending on gate generator).

• r is a coefficient related to the gate generator (often 1 or 1/2).

$\langle M \rangle_{\theta_k+s}$ is the expectation value measured by running the PQC with θ_k shifted by $+s$.

$\langle M \rangle_{\theta_k-s}$ is the expectation value measured by running the PQC with θ_k shifted by $-s$.

Implementation: For each parameter θ_k , we run the PQC twice (once with θ_k+s , once with θ_k-s), measure $\langle M \rangle$, and combine classically.

Pros: Provides exact analytic gradients (not numerical approximations like finite differences, which can be unstable with noise). Can be implemented directly on QPUs.

Cons: Doubles the number of QPU evaluations per parameter per gradient step (if M parameters, $2M$ evaluations for full gradient). Can still be affected by shot noise in estimating $\langle M \rangle$.

Widely used in QML libraries like PennyLane and Qiskit Gradient framework.

Slide 30: Practical Implementation: PennyLane or Qiskit for VQCs

Bringing VQCs to Life with Code:

PennyLane Example (Conceptual):

```
```python
import pennylane as qml
from pennylane import numpy as np

dev = qml.device("default.qubit", wires=num_qubits, shots=1000)
Define data encoding (e.g., AngleEmbedding)
Define variational ansatz (e.g., layers of Rotations and CNOTs)

@qml.qnode(dev)
def quantum_circuit(params, x):
 AngleEmbedding(features=x, wires=range(num_qubits), rotation='Y')
 StronglyEntanglingLayers(weights=params, wires=range(num_qubits))
 return qml.expval(qml.PauliZ(0)) # Measure expval of Z on qubit 0

def cost_function(params, X_batch, Y_batch):
 predictions = [quantum_circuit(params, x) for x in X_batch]
 return mean_squared_error(Y_batch, predictions) # Or other loss

Initialize params
opt = qml.GradientDescentOptimizer(stepsize=0.1)
For epoch in epochs: params = opt.step(cost_function, params, X_train, Y_train)
```
```

Qiskit Machine Learning Example (Conceptual):

```
```python
from qiskit.circuit.library import ZZFeatureMap, RealAmplitudes
```

# Slide 31: Example: VQC for a Simple Classification Task (Conceptual)

**Goal:** Classify 2D data points into two classes (e.g., linearly separable or slightly non-linear).

**Steps (Conceptual, using PennyLane-like flow):**

1. **Generate/Load Data:** `X` (features), `Y` (labels, e.g., 0 or 1, then mapped to -1, +1 for some loss functions).

2. **Define Quantum Device:** `dev = qml.device("default.qubit", wires=num\_qubits, shots=1000)` (e.g., `num\_qubits=2` for 2 features).

3. **Define QNode (Quantum Circuit):**

```
```python
@qml.qnode(dev)
def circuit(weights, x):
    qml.AngleEmbedding(features=x, wires=range(num_qubits), rotation='Y') # Data Encoding
    qml.BasicEntanglerLayers(weights=weights, wires=range(num_qubits)) # Variational Ansatz
    return qml.expval(qml.PauliZ(0)) # Measurement for prediction
```
```

4. **Define Cost Function:**

```
```python
def cost(weights, X_batch, Y_batch):
    predictions = [circuit(weights, x_val) for x_val in X_batch]
    return np.mean((Y_batch - np.array(predictions))**2) # MSE Loss
```
```

5. **Initialize Weights & Optimizer:**

```
`weights = np.random.randn(num_layers, num_qubits, num_params_per_gate)`
`opt = qml.AdamOptimizer(stepsize=0.1)`
```

6. **Training Loop:**

```
`for iter in range(num_iters): weights = opt.step(cost, weights, X_train, Y_train)`
```

7. **Prediction & Evaluation:** Use trained `weights` with `circuit(weights, x\_test)` to predict.

**Interpretation:** The expectation value  $\langle Z \rangle$  (from -1 to +1) is used as a score. A threshold (e.g.,

# Slide 32: Expressivity and Trainability of VQCs/QNNs

## Two Critical, Interrelated Properties:

**Expressivity (Representational Power):** The ability of a PQC ansatz  $U(\theta)$  to generate a diverse range of quantum states (or unitary operations) as its parameters  $\theta$  are varied.

A highly expressive ansatz can, in principle, approximate a wider class of target functions or states.

Measured by how much of the Hilbert space (or group of unitaries) the ansatz can reach.

**Trainability:** The ease and reliability with which a classical optimizer can find good parameters  $\theta$  for the PQC to solve a given task (i.e., minimize the loss function).

## The Expressivity-Trainability Trade-off:

**High Expressivity is Tempting:** Might seem desirable to reach any possible state.

**The Catch:** Very expressive ansätze (especially unstructured, deep ones) are often much harder to train due to:

**Barren Plateaus:** Gradients vanish exponentially with qubit count in highly expressive, deep random circuits.

More complex loss landscapes with many local minima.

**Low Expressivity:** Easier to train (potentially smoother landscapes, fewer parameters), but might not be powerful enough to solve the problem (underfitting).

## Factors Influencing Expressivity & Trainability:

Ansatz architecture (gate choices, entanglement strategy, depth).

Number of qubits.

Problem structure and cost function definition.

Initialization of parameters.

Choice of optimizer.

**Research Focus:** Designing ansätze that are "just right" – expressive enough for the task but structured enough to avoid severe trainability issues. This often involves incorporating problem-specific knowledge or using hardware-efficient structures.



# Slide 33: Barren Plateaus in QNNs and Mitigation Strategies

## The "Gradient Desert": A Major Hurdle for Training Deep/Expressive QNNs.

### What are Barren Plateaus?

Regions in the parameter space of a PQC where the variance of the cost function's gradient ( $\partial C / \partial \theta_k$ ) vanishes exponentially with the number of qubits  $n$ .

This means gradients are, on average, extremely close to zero across most of the landscape.

Optimizer gets no meaningful direction to move; learning stagnates.

### Causes (Still an area of active research):

**Highly Expressive/Random Ansätze:** If the PQC generates states that are too close to "random" (2-designs in group theory terms), gradients tend to vanish.

**Global Cost Functions:** Cost functions that depend on measurements of all or many qubits (e.g., comparing global properties of states) are more prone than local cost functions (depending on few qubits).

**Noise:** Certain types of noise can also induce or exacerbate barren plateaus.

**Entanglement Structure:** Too much "scrambling" entanglement across many layers.

### Mitigation Strategies (Active Research):

#### Careful Ansatz Design:

Using problem-inspired or structured ansätze (e.g., layer-wise learning, shallower circuits, specific entanglement patterns like linear or nearest-neighbor).

Hardware-efficient ansätze that reflect device topology.

**Parameter Initialization Strategies:** Smart initialization might help avoid plateau regions from the start.

**Local Cost Functions:** Designing cost functions that depend on observables of only a few qubits, if possible.

**Layer-wise / Sequential Training:** Train one layer of the PQC at a time.

**Correlating Parameters:** Introducing correlations or constraints between parameters.

**Pre-training or Meta-learning.**

**No Universal Fix Yet:** This remains a significant challenge for scaling QNNs.

# Slide 34: Quantum Principal Component Analysis (qPCA) - Concept & Potential

## Classical PCA Recap:

A dimensionality reduction technique.

Finds principal components (orthogonal directions of largest variance in data).

Projects data onto a lower-dimensional subspace spanned by top principal components, retaining most information.

Involves diagonalizing the covariance matrix of the data.

## Quantum PCA (qPCA) - The Idea:

To perform PCA on quantum states or on classical data encoded into quantum states, potentially faster than classical PCA for certain scenarios.

## One Approach (Lloyd, Mohseni, Rebentrost - 2014):

1. **Input:** Multiple copies of a low-rank density matrix  $\rho$  (representing the data's covariance matrix, or data encoded as a mixed state).
2. **Quantum Task:** Use quantum algorithms (related to phase estimation or matrix exponentiation) to efficiently find the eigenvectors (principal components) and eigenvalues (variances) of  $\rho$ .
3. **Output:** The principal components as quantum states, and their corresponding variances.

## Potential for Exponential Speedup (in dimension $N$ , if...):

If the density matrix  $\rho$  ( $N \times N$ ) can be prepared efficiently.

If we only need to project new data onto a \*few\* top principal components (represented as quantum states).

If the data is genuinely quantum, or if classical data can be efficiently loaded into  $\rho$  (e.g., via QRAM).

## Challenges:

Efficient preparation of the density matrix  $\rho$  from classical data is hard (QRAM problem).

Extracting all classical components of the eigenvectors can be slow.

The "exponential speedup" often refers to the quantum state output; converting to classical can negate gains.

**NISQ Viability:** Full qPCA with exponential speedup is likely beyond NISQ. Simpler, VQA-based approaches

# Slide 35: Quantum k-Means Clustering - Concept & Challenges

## Classical k-Means Clustering Recap:

An unsupervised learning algorithm to partition  $N$  data points into  $K$  clusters.

Iterative process:

1. Initialize  $K$  cluster centroids randomly.
2. Assign each data point to the nearest centroid (based on a distance metric, e.g., Euclidean).
3. Recalculate each centroid as the mean of all points assigned to it.
4. Repeat steps 2-3 until centroids no longer move significantly.

## Quantum k-Means (Qk-Means) - The Idea:

To leverage quantum computation to speed up parts of the k-Means algorithm, particularly the distance calculations or the assignment step.

## Potential Quantum Enhancements:

### 1. Quantum Distance Calculation:

Encode data points and centroids as quantum states.

Use quantum circuits (e.g., based on SWAP test or other overlap measures) to estimate distances (e.g., Euclidean distance squared, Hamming distance) between a data point and all  $K$  centroids, potentially in superposition.

Could offer speedup if  $K$  is large or distance calculation is complex.

### 2. Finding the Nearest Centroid (Grover-like Search):

If distances to centroids can be computed by a quantum oracle, Grover's algorithm could potentially find the index of the nearest centroid for a data point in  $O(\sqrt{K})$  oracle calls, instead of  $O(K)$  classically.

## Challenges:

**Efficient Data Encoding:** Loading  $N$  data points and  $K$  centroids into quantum states.

**QRAM Assumption:** Many theoretical speedups rely on QRAM for accessing data points or centroid information.

**Quantum Distance Oracle Complexity:** The circuit to compute distances quantumly might be complex itself.

# Slide 36: Quantum Boltzmann Machines (QBM) - Generative Models

## Classical Boltzmann Machines (BM) Recap:

A type of stochastic recurrent neural network (an energy-based model).

Used for unsupervised learning (learning a probability distribution over input data) and can be used for generative tasks.

Units (neurons) can be visible (data) or hidden. Connections have weights.

Energy function  $E(v,h)$  defines the system's energy for a given configuration of visible ( $v$ ) and hidden ( $h$ ) units.

Probability of a configuration  $\sim e^{-E(v,h)/T}$  (Boltzmann distribution).

Training is hard (involves sampling, often via MCMC). Restricted Boltzmann Machines (RBMs) are simpler.

## Quantum Boltzmann Machines (QBM) - The Idea:

Replace classical units/probabilities with quantum counterparts (qubits, quantum Hamiltonians, quantum Gibbs states).

Aim to leverage quantum effects (tunneling, entanglement) for potentially better training (sampling) or more expressive generative models.

## Two Main Flavors:

### 1. Quantum Annealer based QBM:

The energy function of a classical BM can be mapped to an Ising Hamiltonian.

A quantum annealer can sample low-energy configurations of this Ising model, which correspond to high-probability samples from the Boltzmann distribution.

Used for training BMs or for direct quantum sampling.

### 2. Gate-Model QBM (Variational):

Use a Parameterized Quantum Circuit (PQC) to prepare a trial quantum state  $|\psi(\theta)\rangle$ .

Train  $\theta$  such that  $|\psi(\theta)\rangle$  approximates the quantum Gibbs state  $(e^{-\beta H}/Z)$  of a target Hamiltonian  $H$  (which might encode the data distribution).

Training involves minimizing a quantum divergence (like quantum relative entropy) or using a quantum-classical hybrid loop.

# Slide 37: Quantum Generative Adversarial Networks (QGANs)

## Classical GANs Recap:

A powerful class of generative models.

Two neural networks trained in competition:

**Generator (G):** Tries to generate realistic data samples (e.g., images) from random noise.

**Discriminator (D):** Tries to distinguish between real data samples and fake samples generated by G.

Training is a minimax game: G tries to fool D, D tries to get better at catching fakes.

Can produce highly realistic synthetic data.

## Quantum GANs (QGANs) - The Idea:

Replace one or both components (Generator, Discriminator) with quantum circuits (PQCs).

Or, process quantum data using a GAN-like framework.

## Possible QGAN Architectures:

**Quantum Generator, Classical Discriminator (qG-cD):** PQC generates quantum states; measurements produce classical samples fed to a classical NN discriminator.

**Classical Generator, Quantum Discriminator (cG-qD):** Classical NN generates data; a PQC acts as a discriminator (e.g., a VQC trained to classify real vs. fake).

**Fully Quantum GAN (qG-qD):** Both generator and discriminator are PQCs. This is more complex, especially defining the "quantum data" and interaction.

## Potential Advantages:

Quantum generators might sample from more complex or "quantum-native" distributions.

Quantum discriminators might be more powerful at distinguishing subtle features.

Could potentially train faster or with fewer parameters for certain tasks.

## Challenges:

Defining appropriate loss functions for the quantum components.

Training stability (classical GANs are already hard to train).

Efficiently loading/representing real (classical) data for a quantum discriminator.

# Slide 38: Quantum Reinforcement Learning - Emerging Ideas

## **Classical Reinforcement Learning (RL) Recap:**

Agent learns to make optimal decisions by interacting with an environment.

Receives states, takes actions, gets rewards/penalties.

Goal: Learn a policy (state  $\rightarrow$  action mapping) that maximizes cumulative reward.

Examples: Game playing (AlphaGo), robotics, control systems.

## **Quantum Reinforcement Learning (QRL) - The Idea:**

Explore how quantum computing might enhance or speed up aspects of RL.

Or, how RL agents might learn to control quantum systems.

## **Potential Areas for Quantum Enhancement in RL:**

### **1. Quantum-Enhanced Agents:**

The agent's policy or value function could be represented by a PQC (a VQC).

Potentially allows for more expressive policies or value functions, or learning with fewer parameters.

### **2. Quantum-Enhanced Environment Simulation:**

If the environment is a quantum system (e.g., controlling a molecule), a quantum computer could simulate it more accurately than a classical one.

### **3. Speedups in RL Algorithms:**

Grover's search or amplitude amplification might speed up parts of some RL algorithms (e.g., searching for optimal actions in a large action space, or improving policy evaluation). This is often highly theoretical.

### **4. Quantum Exploration Strategies:** Using quantum random walks or other quantum dynamics for more efficient exploration of the state-action space.

## **Challenges:**

Defining quantum states, actions, rewards, and policies meaningfully.

Efficiently interfacing a quantum agent with a classical (or quantum) environment.

The QRAM problem for accessing large state/action spaces or experience replay buffers.

Demonstrating clear advantage over sophisticated classical RL algorithms.

# Slide 39: Quantum Natural Language Processing (QNLP) - Recap & Practical Steps

## Session 1 Recap: QNLP Foundations

Language has structure (grammar) and meaning (semantics).

DisCoCat framework: Maps grammatical structure to composition of quantum states (representing word meanings).

Potential for superposition (ambiguity), entanglement (semantic relations), richer vector spaces for meaning.

## Practical QNLP with Libraries like Lambeq (Quantinuum):

1. **Parsing:** Classical sentence  $\rightarrow$  Syntactic Diagram (e.g., using a CCG parser like Bobcat in Lambeq). This diagram captures grammatical structure and word types.
2. **Ansatz Definition:** Define how atomic word types (Noun, Verb, Sentence etc.) map to quantum states/circuits (qubit counts, PQC structure per type).  
e.g., Noun  $\rightarrow$  1 qubit, PQC with 2 parameters. Sentence  $\rightarrow$  1 qubit.
3. **Diagram to Circuit Compilation:** Lambeq converts the syntactic diagram into a (parameterized) quantum circuit by composing the ansatz circuits for individual words according to the grammatical "wiring" in the diagram.
4. **Model Training (Hybrid):**  
The parameters in the word ansätze are the learnable weights.  
Train for a task (e.g., text classification, sentiment analysis) using a classical optimizer and a loss function, with the QPU/simulator executing the sentence circuits.

## Example Task: Sentiment Analysis

Sentence circuit output (e.g., expectation value)  $\rightarrow$  mapped to sentiment score/class.

**Challenges:** Scalability to large vocabularies/sentences, designing effective word ansätze, hardware noise, demonstrating advantage over classical NLP (BERT, GPT).

**Focus:** Compositional semantics, exploring if quantum structure aids meaning representation.

# Slide 40: Quantum Image Processing (QIP) - Recap & Practical Steps

## Session 1 Recap: QIP Foundations

Images are high-dimensional data (pixels, colors).

QIP aims to use quantum mechanics for image representation, storage, and processing.

Quantum Image Representations (QIRs): FRQI, NEQR – aim for compact encoding of pixel position & value using superposition.

Potential for parallelism in global operations.

## Practical QIP Steps (Conceptual):

1. **Choose a QIR:** e.g., NEQR (Novel Enhanced Quantum Representation) for its direct grayscale encoding.

$|I\rangle = (1/2^{\lceil n \rceil}) \sum_{y,x} |Color(y,x)\rangle \otimes |yx\rangle$  (position `yx`, color `Color(y,x)`).

2. **Quantum Image Preparation (The Bottleneck):**

Design a circuit to load the classical pixel data into the chosen QIR state.

This is generally complex and resource-intensive for arbitrary images.

3. **Apply Quantum Image Processing Operation  $U_{op}$ :**

Design a quantum circuit  $U_{op}$  that performs the desired task (e.g., negation, filtering, edge detection, geometric transformation) by acting on the QIR state  $|I\rangle$ .

Output state:  $|I'\rangle = U_{op}|I\rangle$ .

4. **Image Readout/Measurement:**

Extract classical image information from  $|I'\rangle$ .

Measuring all pixel values is generally infeasible (requires many measurements or full tomography).

Often, algorithms aim to extract specific features or global properties.

## Example Task: Image Edge Detection (Conceptual)

QIR + circuits to compare neighboring pixel values (compute gradients) in superposition.

Measure parts of the state corresponding to high gradient values.

**Challenges:** Efficient QIR preparation & readout, designing complex  $U_{op}$  circuits, noise,



# Slide 41: Choosing the Right QML Algorithm for a Problem

**No "One QML Algorithm Rules Them All":** The best choice depends on the problem, data, and available quantum resources.

## **Factors to Consider:**

### **1. Problem Type:**

**Classification?** Consider QSVM (especially if a good quantum kernel is hypothesized) or VQC/QNN.

**Generative Task?** Explore QBMs, QGANs (more experimental).

**Dimensionality Reduction?** qPCA (theoretical, QRAM-dependent) or variational quantum autoencoders (NISQ-friendlier).

**Optimization within an ML context?** QAOA or quantum annealers for sub-problems like feature selection.

### **2. Data Characteristics:**

**Data Size ( $N_{\text{samples}}$ ,  $N_{\text{features}}$ ):** Affects kernel matrix computation time (QSVM), data loading feasibility, qubit requirements for encoding.

**Data Structure/Nature:** Is there inherent quantum structure? Are there known symmetries? This might guide feature map design. Is data linearly separable or highly non-linear?

### **3. Potential for Quantum Advantage:**

Does the problem have a structure where a known quantum primitive (e.g., QFT, Grover) could offer a theoretical speedup if integrated?

Is the goal to explore if a quantum feature space offers better representation than classical ones (kernel methods)?

### **4. NISQ Feasibility:**

**Circuit Depth & Qubit Count:** Can the algorithm be implemented with shallow enough circuits on available qubits?

**Noise Resilience:** How sensitive is the algorithm to hardware noise? VQAs are often preferred.

# Slide 42: Data Preprocessing for QML

**Classical Preprocessing Still Essential (and sometimes more so!):** Just like in classical ML, data preprocessing is a vital step before feeding data into a QML algorithm.

## **Common Preprocessing Steps Relevant to QML:**

### **1. Feature Scaling / Normalization:**

Many quantum encoding schemes (especially angle encoding) expect features to be within a specific range (e.g.,  $[0, \pi]$ ,  $[-1, 1]$ ).

Classical techniques: Min-Max scaling, Standardization (Z-score normalization).

Amplitude encoding requires the feature vector to be L2-normalized (sum of squares = 1).

### **2. Dimensionality Reduction (Classical):**

If you have very high-dimensional classical data, it might be beneficial to first use classical dimensionality reduction techniques (e.g., PCA, t-SNE, autoencoders) to reduce the number of features *\*before\** encoding them into qubits.

This helps manage qubit requirements and can make the QML task simpler.

### **3. Handling Categorical Features:**

Convert to numerical representations (e.g., one-hot encoding, embedding layers) before quantum encoding. Basis encoding can handle a few categories directly.

**4. Data Cleaning:** Handling missing values, removing outliers (as these can disproportionately affect some quantum encodings or distance measures).

## **Why it's Extra Important for QML:**

**Qubit Limitations:** We have very few qubits, so efficient use of features is paramount.

**Sensitivity of Encodings:** Some quantum encoding schemes can be sensitive to the scale and distribution of input features.

**Cost of Quantum Computation:** We want to ensure the data fed to the QPU is as "clean" and informative as possible to make the most of expensive quantum resources.

**No Magic Bullet:** QML won't magically fix poorly preprocessed data. Standard ML best practices apply.

# Slide 43: Software Tools & Libraries for QML (Qiskit, PennyLane Focus)

## Your Toolkit for QML Development:

### 1. Qiskit (IBM):

#### ``qiskit_machine_learning`` module:

**Data Encoding/Feature Maps:** ``PauliFeatureMap``, ``ZZFeatureMap``, ``NLocal`` for custom maps.

**Quantum Kernels:** ``QuantumKernel`` class for computing kernel matrices.

**Classifiers:** ``QSVC`` (Quantum Support Vector Classifier), ``VQC`` (Variational Quantum Classifier), ``NeuralNetworkClassifier`` (for PQC-based models).

**Regression:** ``VQR`` (Variational Quantum Regressor).

**Distributions:** ``QuantumGenerativeAdversarialNetwork`` (QGAN) (experimental).

Integrates with Qiskit's core (Terra for circuits, Aer for simulation, optimizers from ``qiskit_algorithms.optimizers``).

Direct execution on IBM Quantum hardware.

### 2. PennyLane (Xanadu):

**Quantum Differentiable Programming Focus:** Ideal for gradient-based VQAs/QNNs.

**Data Encoding Templates:** ``AngleEmbedding``, ``AmplitudeEmbedding``, ``BasisEmbedding``.

**Variational Circuit Templates (Layers):** ``BasicEntanglerLayers``, ``StronglyEntanglingLayers``, ``RandomLayers``, etc., for building ansätze.

**Measurement Functions:** ``qml.expval()``, ``qml.probs()``, ``qml.var()``.

**Built-in Optimizers:** ``GradientDescentOptimizer``, ``AdamOptimizer``, ``AdagradOptimizer``, etc. (leverage autograd for quantum gradients).

**Interfaces with PyTorch, TensorFlow, JAX:** Allows building hybrid models where quantum circuits are layers within classical deep learning frameworks.

**Broad Hardware/Simulator Support via Plugins.**

**Other Tools:** TensorFlow Quantum (TFQ - integrates Cirq with TensorFlow), Microsoft QDK.

**Key:** These libraries abstract many low-level details, allowing focus on algorithm design and experimentation.

# Slide 44: Running QML on Simulators vs. Real Hardware: What to Expect

## Simulators: Your First Port of Call

### Pros:

- Fast for small qubit counts (< ~20-25 qubits for complex circuits).
- Full access to statevector (for debugging, perfect probabilities/expvals).
- Noise modeling capabilities (can simulate different error channels).
- Free, accessible, repeatable.

### Cons:

- Classical simulation scales exponentially with qubits (intractable > ~40-50 qubits).
- Idealized noise models may not perfectly capture real hardware intricacies.

**Use For:** Algorithm development, debugging, initial hyperparameter tuning, understanding ideal performance, small-scale QML experiments.

## Real Quantum Hardware (QPUs): The True Test

### Pros:

- The only way to test "true" quantum effects and potential for advantage on problems intractable for simulators.
- Experience real-world device characteristics (specific noise, topology, gate sets).
- Contributes to the collective learning about current hardware capabilities.

### Cons:

- Noise:** Decoherence, gate errors, readout errors will significantly impact results.
- Limited Qubits & Connectivity:** Restricts the size and complexity of QML models.
- Queue Times & Cost:** Access can involve waiting and may have associated costs.
- Calibration Drifts:** Device performance can vary over time.
- No Access to Full Statevector:** Only get probabilistic measurement outcomes.

**Use For:** Benchmarking against simulators, testing noise resilience, error mitigation studies, exploring potential quantum advantage, research on hardware-specific QML.

**Typical Workflow:** Develop & Debug on Simulator → Test & Refine on QPU with Error Mitigation.

# Slide 45: Benchmarking QML Models: Against Classical & Other Quantum Models

## A Crucial Step: How Good is Your QML Model?

### 1. Benchmarking Against Classical Baselines:

**The Most Important Comparison:** Can your QML model outperform (or at least match) well-established classical ML algorithms on the same task and dataset?

**Classical Algorithms to Compare Against:**

For classification: Logistic Regression, Classical SVM (with various kernels like Linear, RBF, Poly), Decision Trees/Random Forests, Gradient Boosting Machines (XGBoost, LightGBM), Neural Networks (MLPs, CNNs for image data, RNNs/Transformers for sequential data).

**Metrics:** Accuracy, Precision, Recall, F1-score, AUC-ROC, training time, inference time, model size/parameters.

**Fair Comparison:** Ensure classical models are well-tuned (hyperparameter optimization). Use same train/test splits.

**The Goal:** Demonstrate a clear, practical advantage (not just novelty).

### 2. Benchmarking Against Other QML Models:

Comparing different quantum feature maps.

Comparing different ansatz architectures for VQCs.

Comparing different quantum kernel estimation techniques.

Comparing performance on simulators vs. different QPUs.

### 3. Ablation Studies (for QML models):

Systematically removing or altering components of your QML model (e.g., entanglement layers in ansatz, specific encoding features) to understand their contribution to performance.

## Challenges in QML Benchmarking:

Small dataset sizes often used in QML research (due to simulation/QPU limits) may not be representative for classical SOTA.

Difficulty in fair comparison of resource costs (QPU time vs. CPU/GPU time).

Defining "advantage": Is it accuracy, speed (training/inference), fewer parameters, better

# Slide 46: Interpreting Results from QML Models

## Beyond Accuracy: Understanding "Why" and "How"

### Challenges in Interpretability for QML (similar to some classical Deep Learning):

**"Black Box" Nature:** Complex PQCs (especially with deep ansätze or intricate feature maps) can be hard to interpret. It's not always clear *\*why\** the model makes a particular prediction based on its learned parameters  $\theta$ .

**High-Dimensional Feature Spaces:** If a quantum feature map projects data into a very large, abstract Hilbert space, it's difficult to visualize or intuitively understand what features the model is "seeing" or how the decision boundary is formed in that space.

**Impact of Noise:** Real hardware noise can obscure the "true" learned function, making interpretation even harder.

### Techniques for Gaining Insights (Exploratory):

#### 1. Feature Importance (Indirect):

If using angle encoding, one could try to analyze the sensitivity of the model's output to small changes in individual input features (similar to classical gradient-based feature importance, but gradients are w.r.t. data inputs here, which is harder).  
Ablation: Systematically remove or mask input features and observe impact on performance.

#### 2. Visualizing Learned States (for very small qubit systems):

For VQCs with few qubits, one might try to visualize the trajectory of the quantum state on the Bloch sphere (for 1 qubit) or using other state representations as parameters are learned, or for different input data. Highly limited.

#### 3. Analyzing Learned Parameters $\theta$ :

Are there patterns in the optimized  $\theta$  values? Do certain gates/rotations consistently get large/small angles? This is usually very hard to connect directly to decision logic.

#### 4. Focus on Observable Outputs:

Relate the expectation values  $\langle M \rangle$  being measured to understandable properties or decision thresholds.

# Slide 47: Error Mitigation Techniques in QML Experiments

**NISQ Reality: Noise is a Given.** We can't fully correct errors yet, but we can try to *\*mitigate\** their impact on results.

**Goal of Quantum Error Mitigation (QEM):** To estimate the ideal, noise-free expectation value (or probability) from many noisy measurements obtained from a QPU.

**Common QEM Techniques Relevant for VQCs/QML:**

**1. Readout Error Mitigation:**

Corrects for errors that occur during the final measurement process (e.g., a qubit in state  $|1\rangle$  being misread as  $|0\rangle$ ).

Involves a calibration step: prepare all basis states ( $|00\rangle$ ,  $|01\rangle$ , ...), measure them, and build a "confusion matrix" or response matrix.

This matrix is then used to classically post-process raw measurement counts to get a more accurate estimate of the true probability distribution.

**2. Zero-Noise Extrapolation (ZNE):**

Run the quantum circuit at different, controlled levels of noise (e.g., by artificially increasing gate durations or inserting identity gate pairs to amplify existing noise).

Measure the expectation value for each noise level.

Extrapolate the results back to the zero-noise limit.

**3. Probabilistic Error Cancellation (PEC):**

Requires a detailed model of the noise (gate errors).

Learns to "invert" the noise channel by running additional "quasi-probability" circuits that effectively cancel out the dominant noise terms on average.

Can be resource-intensive (many more circuit executions).

**4. Gate Set Tomography / Randomized Benchmarking:** Techniques to characterize gate errors accurately, which can inform PEC or other mitigation methods.

**Integration with QML:** These QEM techniques are often applied when estimating the cost function  $C(\theta)$  or the final prediction from a VQC.

**Trade-off:** QEM usually requires more measurements or circuit executions (overhead) but can significantly

# Slide 48: Current Bottlenecks for Practical QML Advantage

**Recap: The Path to Advantage is Steep.**

## **1. Hardware Limitations (NISQ):**

**Qubit Count & Quality:** Insufficient number of high-fidelity, well-connected qubits for large, complex problems/datasets.

**Noise Levels:** Decoherence, gate errors, readout errors fundamentally limit circuit depth and accuracy.

## **2. Data Encoding & Loading Bottleneck:**

Efficiently preparing quantum states that meaningfully represent large classical datasets is a major unsolved problem (QRAM).

Cost of state preparation can dominate any downstream quantum speedup.

## **3. Algorithm Trainability & Optimization:**

**Barren Plateaus:** Vanishing gradients hinder training of expressive PQCs.

**Cost Function Evaluation:** Estimating expectation values/kernels requires many shots, making each step of VQA optimization slow.

Classical optimizers may struggle with noisy, high-dimensional QML landscapes.

## **4. Lack of "Killer" QML Algorithms with Proven End-to-End Speedups for Real Problems:**

Many QML algorithms offer theoretical speedups for subroutines, but overall practical advantage (including data I/O, noise) is hard to achieve and prove against highly optimized classical SOTA.

Need for QML algorithms that are not just "quantum versions" of classical ones, but truly leverage unique quantum features for a task.

## **5. Benchmarking Challenges:**

Fair comparison with classical ML (which benefits from decades of optimization and massive datasets).

Defining appropriate metrics for "advantage" (accuracy, speed, data efficiency?).

**Overcoming these requires progress on multiple fronts: hardware, algorithms, theory, and software.**



# Slide 49: Promising Application Areas for Near-Term QML

**Where Might NISQ QML Make an Impact Sooner?** Focus on problems where quantum representations or processing might offer a unique edge, even with limited resources.

**1. Materials Science & Drug Discovery (via VQE-like approaches for property prediction):**

Using QML to learn relationships between molecular structure (encoded quantumly) and properties (energy, reactivity).

The "data" itself (molecular Hamiltonians) is inherently quantum.

VQE is a core tool; QML can help analyze VQE outputs or guide experiments.

**2. Enhancing Classical ML with Quantum Kernels (for specific datasets):**

If a dataset exists where a quantum feature map can find better separability than any known classical kernel, QSVM could provide an advantage in accuracy.

Identifying such datasets and "killer" feature maps is key.

**3. Optimization Problems in ML (Feature Selection, Hyperparameter Tuning):**

Using QAOA or quantum annealers to solve combinatorial optimization sub-problems within a larger ML workflow.

Example: Finding the optimal subset of features for a classical model.

**4. Generative Modeling for Specific Niche Distributions:**

QBMs or QGANs might be able to learn and sample from certain types of complex probability distributions that are hard for classical models, perhaps those with quantum-like correlations.

Applications in finance (modeling market data) or physics (simulating quantum systems).

**5. Learning from Quantum Data:**

If the input data itself comes from quantum experiments or quantum sensors, QML algorithms might be the most natural and efficient way to process and learn from it.

**Key: Look for problems where classical methods struggle AND where quantum mechanics offers a natural or fundamentally different approach to representation/computation.**

# Slide 50: The Role of Co-design (Hardware, Software, Algorithm) in QML Progress

**No Single Breakthrough Will Suffice:** Progress in QML (and QC in general) relies on synergistic advancements across multiple layers.

## **Hardware-Software-Algorithm Co-design:**

**Hardware Development:** Building more qubits, higher fidelity gates, longer coherence times, better qubit connectivity. Different hardware platforms (superconducting, trapped ion, photonic, neutral atom) have different strengths and weaknesses.

**Software & Compiler Development:** Creating more efficient quantum compilers that map algorithms to specific hardware optimally (minimizing SWAPs, gate depth). Developing better noise models and error mitigation techniques integrated into software. Building user-friendly SDKs.

**Algorithm Design:** Developing new QML algorithms that are inherently noise-resilient, require fewer resources, or are specifically tailored to the strengths of a particular hardware architecture ("hardware-aware" algorithms).

## **The Feedback Loop:**

New hardware capabilities enable more complex algorithms to be tested.

Experimental results from algorithms inform hardware improvements and compiler optimizations.

Theoretical algorithmic insights can guide what hardware features are most desirable.

## **Example: Hardware-Efficient Ansätze for VQCs:**

Ansatz structures are designed based on the native gates and connectivity of a specific QPU to minimize errors and circuit depth.

This is a direct result of co-design.

**Importance for QML:** Given the NISQ constraints, QML algorithms that are co-designed with specific hardware in mind are more likely to show practical utility.

**A Holistic Approach is Needed for the Field to Advance.**

# Slide 51: The Path to Demonstrable Quantum Advantage in ML

**The "Holy Grail" of QML:** Achieving a clear, practical, and widely accepted quantum advantage for a real-world machine learning task.

**What Would "Advantage" Look Like?**

**Better Performance:** Significantly higher accuracy, precision, recall, F1-score, or other relevant ML metric than the best classical SOTA on the same data.

**Faster Training/Inference:** Substantial speedup (e.g., polynomial or exponential) for training a model to a certain performance level, or for making predictions with a trained model, including all overheads (data I/O, QPU time, classical processing).

**Better Generalization from Less Data:** Ability to train high-performing models with significantly smaller training datasets than classical counterparts.

**Solving Previously Intractable ML Problems:** Enabling solutions to ML tasks that are currently computationally infeasible for classical methods.

**More Compact Models:** Achieving comparable performance with far fewer learnable parameters.

**Key Ingredients Still Needed for Demonstrable Advantage:**

1. **Improved Quantum Hardware:** More high-fidelity qubits, longer coherence, better connectivity, lower error rates (progress towards FTQC).
2. **"Killer" QML Algorithms/Applications:** QML algorithms that are not just quantum analogues of classical ones but truly exploit quantum mechanics for a specific ML task in a way that provides a fundamental edge.
3. **Efficient Data Handling:** Solutions to the data loading (QRAM-like capabilities or very efficient encoding for specific data) and readout bottlenecks.
4. **Robust & Scalable Training Techniques:** Overcoming barren plateaus, developing efficient quantum gradient methods, and robust classical optimizers for noisy QML landscapes.
5. **Rigorous Benchmarking & Fair Comparisons:** Against highly optimized classical state-of-the-art, on relevant, large-scale datasets.

**It's a Marathon, Not a Sprint:** Requires sustained, collaborative effort.

# Slide 52: Ethical Considerations and Responsible QML Development

**With Great Power Comes Great Responsibility:** As QML (and QC in general) advances, it's crucial to consider the ethical implications and promote responsible development.

**Potential Societal Impacts (Shared with General AI & QC):**

**Job Displacement:** Automation of tasks currently done by humans (though QML is far from this stage for broad applications).

**Bias and Fairness:** QML models, like classical ML, can inherit and amplify biases present in training data. Ensuring fairness in quantum algorithms and their outputs is critical. How do we audit "black box" quantum models for bias?

**Security Implications (from general QC, impacting ML data security):** Shor's algorithm's threat to current encryption could impact the security of sensitive data used in ML. The need for post-quantum cryptography.

**Access and Equity:** Ensuring that the benefits of QML (if realized) are accessible broadly and do not exacerbate existing inequalities (the "quantum divide").

**Misuse of Powerful AI:** Generative QML models (if they become very powerful) could potentially be misused, similar to concerns with classical deepfakes or disinformation.

**Specific QML Considerations:**

**Interpretability & Explainability:** As QML models become more complex, understanding their decision-making process is vital for trust, accountability, and debugging, especially in critical applications (e.g., medicine, finance).

**Environmental Impact:** While individual quantum operations might be low energy, the overall energy footprint of large-scale quantum data centers (cooling, classical control) needs consideration.

**Guiding Principles for Responsible QML:**

Transparency in development and limitations.

Focus on beneficial applications.

Proactive research into fairness, accountability, and transparency (FAT/XAI for QML).

Inclusive development involving diverse stakeholders.

# Slide 53: Summary of Practical QML & Key Learnings from Session 2

## Session 2 Recap: Making QML More Concrete

### Data Encoding is Foundational:

Quantum Feature Maps (Angle, Amplitude, PQC-based) translate classical data.

Choice impacts resources, performance, and potential for advantage.

Data loading (QRAM problem) remains a major hurdle for some approaches.

### Quantum Kernel Methods (e.g., QSVM):

QPU estimates kernel  $K_{ij} = |\langle \Phi(x_i) | \Phi(x_j) \rangle|^2$ .

Classical SVM uses this quantum kernel for training/prediction.

Advantage hinges on "hard-to-simulate-classically" rich feature spaces.

### Variational Quantum Classifiers (VQCs/QNNs):

PQC (Data Encoding + Variational Ansatz  $U(\theta)$ ) is the trainable model.

Hybrid loop: QPU (PQC eval, measurement) + CPU (loss, gradient,  $\theta$  update).

Parameter-shift rule for analytic gradients.

Expressivity vs. Trainability (Barren Plateaus) is a key design challenge.

**Other QML Paradigms:** qPCA, Qk-Means, QBMs, QGANs, QRL, QNLP, QIP – mostly exploratory, each with unique potentials and significant challenges.

**Practical Workflow:** Preprocess Data → Choose Encoding → Design/Select QML Algorithm → Train (Hybrid) → Benchmark Rigorously → Interpret (if possible) → Mitigate Errors.

**NISQ Constraints Dominate Current Practicality.**

# Slide 54: Final Q&A and Thank You

## Questions, Thoughts, and Future Directions?

Thank you for joining this 2-part exploration of Quantum Machine Learning Foundations and Practical Aspects!

**Key Message:** QML is a rapidly evolving, interdisciplinary frontier with immense theoretical potential and significant near-term challenges.

**Encouragement:** Stay curious, keep learning, explore the tools, and consider how you might contribute!

[Your Contact Email/Website (Optional)]

[Relevant QML Community/Resource Links (Optional)]

